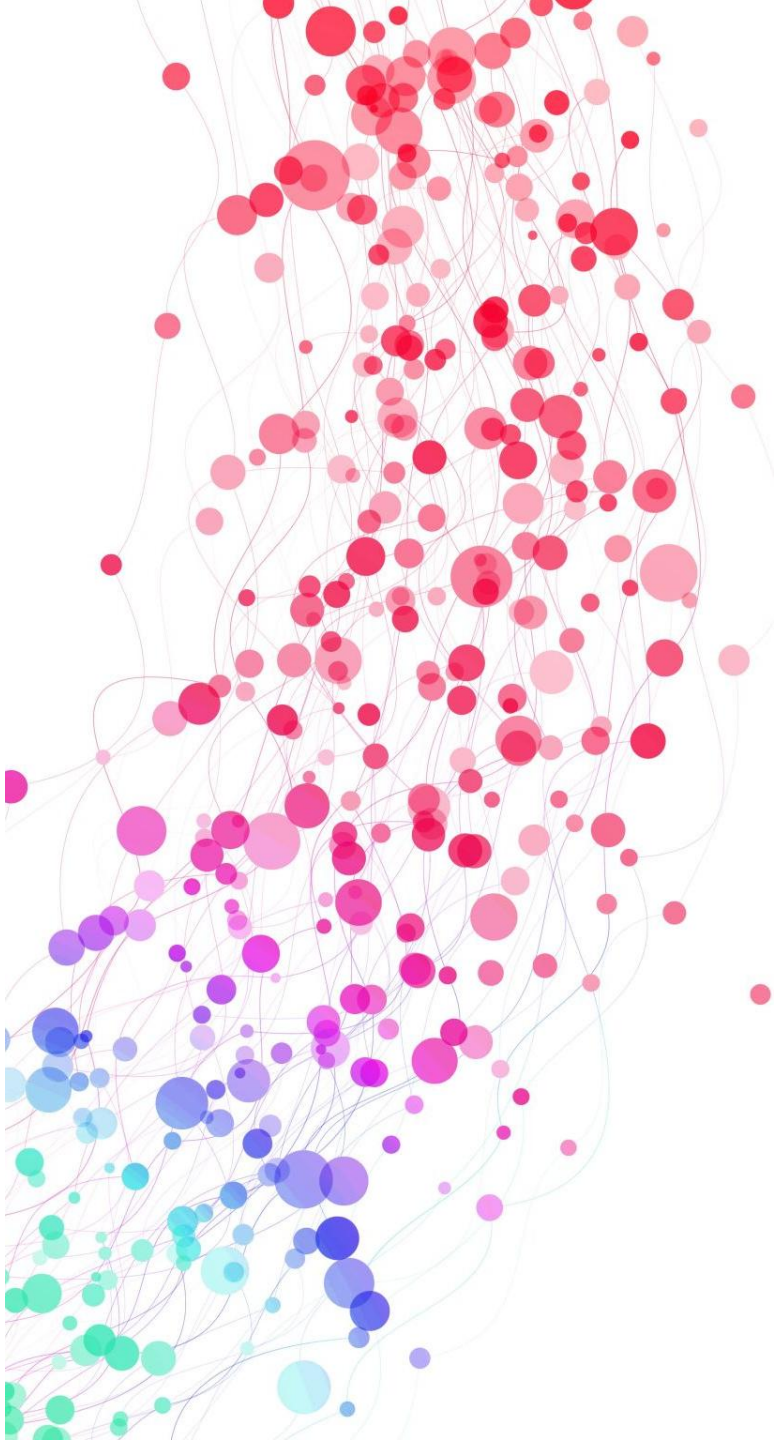
LECTURE 3

Computer programming II



REMEMBER

constructor:

- A constructor is a member function “has the same name of class”.
- It hasn’t a return data type, as it Cannot return anything, not even **void, it** implicitly "returns" the new object being created.
- It executed or run automatically when the client declares a new object of the class.
- Constructors can refer to the object's fields.
- Sets the initial state of each new object.
- Often accepts parameters for the initial state of the fields.
- If a class has no constructor, C++ gives it a default constructor with no parameters that does nothing.
- A class may define more than one *constructor.*
- We can define constructor outside class definition using scope resolution operator.

Compiler provides one if you do not!

Compiler’s default simply calls constructors of data members of the class.

Constructor Types:

1. Default constructor “constructor without any parameter”

```
class Counter
{
private:
    unsigned int count; //count
public:
    Counter():count(0)  //constructor
    { count++; }
    int get_count() //return count
    { return count; }
};
int main(){
    Counter c1, c2; //define and initialize
    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count() <<endl;

    system("Pause");
}
```

Constructor Types:

2. Parameterized constructor “constructor with parameters” used for initialize data member

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)  //constructor (no args)
    { }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
};

int main(){
    Distance dist1, dist3; //define two lengths
    Distance dist2(11, 6.25); //define and initialize dist2

    system("Pause");
}
```

Constructor Types:

3. Copy constructor "used when 1 object of class initialize another object of the same class"

```
class Distance
{
private:
    int feet;
    float inches;
public:
    Distance() : feet(0), inches(0.0)  //constructor (no args)
    { }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft), inches(in)
    { }
};

int main(){
    Distance dist1(11, 6.25); //two-arg constructor
    Distance dist2(dist1); //one-arg constructor
    Distance dist3 = dist1; //also one-arg constructor

    system("Pause");
}
```

Default constructors

- A constructor whose parameters all have a default argument is considered a default constructor.
- It would be an error to create a constructor that accepts no parameters along with another constructor that has default arguments for all its parameters. compiler don't able to resolve which constructor to execute.

```
Circle(){}//default constructor defined by compiler
```

```
Circle(){//default constructor with default argument
  radius=1;
  color="red";
}
```

Error: 'Circle()' cannot be overloaded with Circle().

Classes with no default constructors

- If the class has an overloaded constructor, then the compiler doesn't provide the default constructor.
- It is the responsibility of the programmer to provide the default one.

```
#include <iostream>
using namespace std;
class Circle
{
  private:
    double radius;
  public:
    Circle(double r)
    {
       radius = r;
    }
    double getArea()
    {
      return radius * radius * 3.14;
    }
    double getCircumference()
    {
      return 2 * radius  * 3.14;
    }
};
```

```
int main()
{
  Circle c1; //ERROR: cant find Circle() constructor
  Circle c2(5); //invoke overload constructor
  cout<<"Area1: "<<c1.getArea()<<endl;
  cout<<"Area2: "<<c2.getArea()<<endl;
}
```

Error???

If the class has an overloaded constructor, then the compiler doesn't provide the default constructor.

More about default constructor

- This constructor has default arguments for each of its parameters. As a result, the constructor can be called with no arguments:

```
Rectangle(){ //default constructor with default arguments
     width=1;
     length=1;
}
Rectangle(double w, double l){ //overloaded constructor
     width=w;
     length=l;
}
```

```
Rectangle r2;
Rectangle r1(5,6);
```

```
Rectangle(double w=1, double l=1){
     width=w;
     length=l;
}
```

The keyword this:

C++ has a `this` keyword to refer to the current object.

– *Common usage:* In constructor, so parameter names can match the names of the object's member variables:

```
BankAccount::BankAccount(string name,
                         double balance)
{
    this->name = name;
    this->balance = balance;
}
```

Note:-

this uses -> not . because it is a "pointer"; we'll discuss that later

```
class Employee {
private:
    // Private attribute
    int salary;
public:
    // Setter
    // Precondition: salary is non-negative
    void setSalary(int salary) {
        if (salary<0){
            cout<<"Salray can't ne negative";
        }
        else{
            this-> = salary;
        }
    }
    // Getter
    int getSalary() const{
        return salary;
```

getSalary
salary — private : int Employee::salary
Private attribute
File: Source.cpp
setSalary

```
void setSalary(int salary) {
    if (salary<0){
        cout<<"Salray can't ne negative";
    }
    else{
        this->salary = salary;
    }
}
```

DESTRUCTOR

Destructor:

- A destructor is a member function that has the same name of the class as the constructor but is preceded by a tilde ~
- It used to clean up an object of a class prior to deleting that object.
- It has No parameters, no result
- It is called automatically when an object is destroyed, when the function exits scope of automatic class object
- We destroyed the object from memory using **delete** operator to release the storage that is no longer accessible.
- **Purpose of destructors:-**
  - Free dynamic storage pointed to only by members of object
  - Reduce reference count when object disappears
  - Safely close things – e.g., files

```
~Distance(){   //destructor

}
```

Compiler provides one if you do not!

Mutators and Accessors
"Setter & getter"

Accessor (getter) and mutator (setter) functions

- Accessing data members of a class depends upon the access specifiers of these members (public data).
- Sometimes there is a necessity to provide access even to private data members using **accessor** (getter) and mutator (setter) functions.

Accessor (getter) function

- **Accessor function** is used to get the value of a private data member.
- To properly name this function, begin with "get" followed by data member identifier.
- The return type must match the data member type.

```
double getRadius()
{
    return radius;
}
```

Mutator (setter) functions

- **Mutators** are used to set values of private data members.
- One of the main goals of a mutator is to check correctness of the value to be set to data member.
- A setter's name starts with **set**, followed by **name of data member**.
- Setter must have one parameter of the same type as the data member to bet set by the setter.

```
Void setRadius(double r)
{
  radius=r;
}
```

Example: Circle class

Circle
- radius: double - color: string
+ Circle() + Circle(double) + getRadius(): double + setRadius(double):void + setColor(string):void + getColor(): string + getArea(): double + getCircumference(): double

Class Definition

Circle
-radius:double=1.0 -color:String="red"
+Circle() +Circle(r:double) +Circle(r:double,c:String) +getRadius():double +getColor():String +getArea():double

Instances

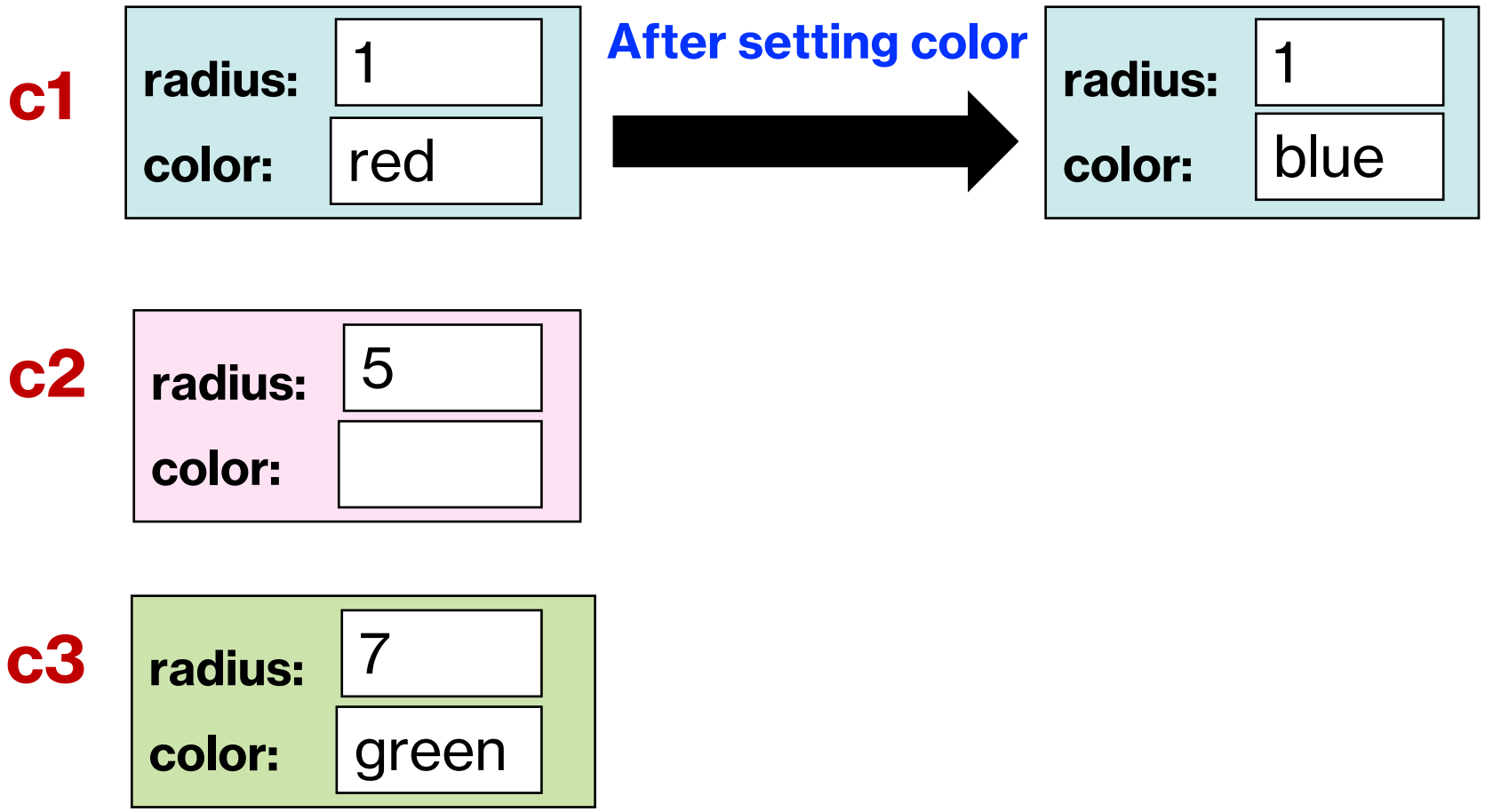
c1:Circle	c2:Circle	c3:Circle
-radius=2.0 -color="blue"	-radius=2.0 -color="red"	-radius=1.0 -color="red"
+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()	+getRadius() +getColor() +getArea()

```
#include <iostream>
using namespace std;
class Circle
{
  private:
    double radius;
    string color;
  public:
    Circle() {
       radius = 1.0;
       color="red";
    }
    Circle(double r){
       radius = r;
    }
    Circle(double r, string c){
       radius = r;
       color=c;
    }
    double getArea(){
      return radius*radius*3.14;
    }
    double getCircumference(){
      return 2 *radius * 3.14;
    }
    double getRadius(){
      return radius;
    }
```

```
color1: red
color1: blue
Area1: 3.14
Area2: 78.5
color3: green
```

```
    void setRadius(double r){
      radius=r;
    }
    string getColor(){
      return color;
    }
    void setColor(string c){
      color=r;
    }
};
int main()
{
  Circle c1; //invoke default constructor
  Circle c2(5); //invoke overload constructor
  Circle c3(7,"green");//overload constructor
   cout<<"color1: "<<c1.getColor()<<endl;
  c1.setColor("blue");
  cout<<"color1: "<<c1.getColor()<<endl;
  cout<<"Area1: "<<c1.getArea()<<endl;
  cout<<"Area2: "<<c2.getArea()<<endl;
  cout<<"color3: "<<c3.getColor()<<endl;
}
```

Circle objects in memory



```cpp
class Employee {
private:
    // Private attribute
    int salary;

public:
    // Setter
    void setSalary(int s) {
        salary = s;
    }
    // Getter
    int getSalary() const{
        return salary;
    }
};
int main(){
    Employee myObj;
    myObj.setSalary(50000);
    cout << myObj.getSalary();
    system("Pause");
}
```

- **Mutator(Setter)**:
  a member function that stores a value in a private member variable, or changes its value in some way

- **Accessor(getter)**:
  function that retrieves a value from a private member variable. **Accessors do not change an object's data**, so they should be marked `const`.

const keyword appearing after the parentheses in a member function declaration specifies that the function will not change any data in the calling object.

precondition: Something your code *assumes is true* at the start of its execution.

- Often documented as a comment on the function's header:

```
// Initializes a BankAccount with the
  given state.
  // Precondition: balance is non-negative
  BankAccount::BankAccount(string name,
  double balance) {
      this->name = name;
      this->balance = balance;
  }
```

- Stating a precondition doesn't really "solve" the problem, but it at least documents our decision and warns the client what not to do.

- What if we want to actually enforce the precondition

```
class Employee {
private:
    // Private attribute
    int salary;
public:
    // Setter
    // Precondition: salary is non-negative
    void setSalary(int s) {
        if (s<0){
            cout<<"Salray can't ne negative";
        }
        else{
            salary = s;
        }
    }
    // Getter
    int getSalary() const{
        return salary;
    }
};
```

Replacement of access specifiers

- There is no rule requiring you to declare private members before public members. The class can be declared as:

```
class Circle
{
   private:
     double radius;
     string color;
   public:
     double getArea(){
       return radius * radius * 3.14;
     }
     double getCircumference(){
       return 2 * radius  * 3.14;
     }
};
```

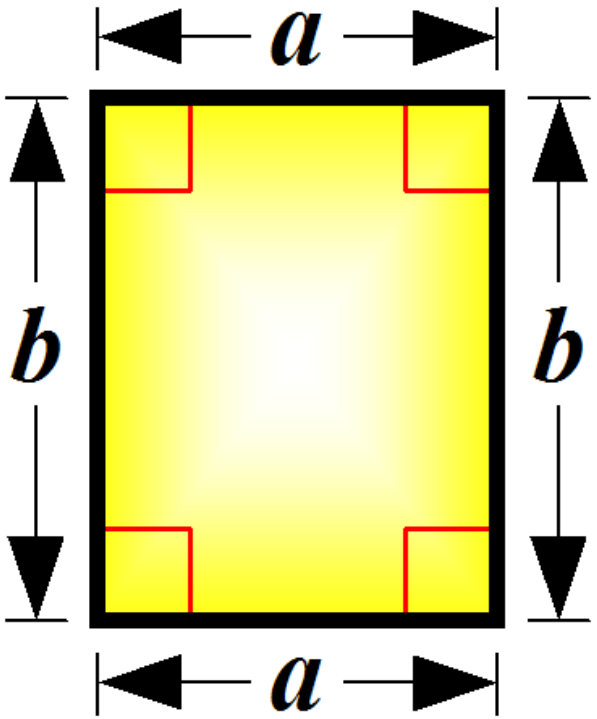
```
class Circle
{
 public:
     double getArea(){
       return radius * radius * 3.14;
     }
     double getCircumference(){
       return 2 * radius  * 3.14;
     }
  private:
     double radius;
     string color;
};
```

Replacement of access specifiers

- It is not required that all members of the same access specification be declared in the same place:

```
class Circle
{
  public:
     double getArea(){
       return radius * radius * 3.14;
     }
  private:
     double radius;
  public:
     double getCircumference(){
       return 2 * radius  * 3.14;
     }
  private:
     string color;
};
```

EXAMPLES



1- Rectangle

Rectangle
- width : double - length : double
+ setWidth(w : double) : void + setLength(len : double) : void + getWidth() : double + getLength() : double + getArea() : double

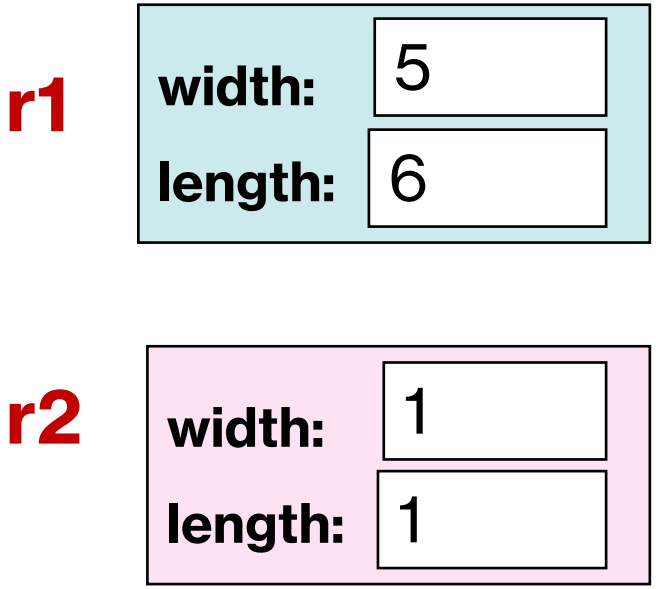
Rectangle

```
#include <iostream>
using namespace std;
class Rectangle{
  private:
    double length;
    double width;
  public:
    Rectangle(){ //default constructor with default arguments
     width=1;
     length=1;
   }
   Rectangle(double w, double l){ //overloaded constructor
     width=w;
     length=l;
   }
   double getLength(){
     return length;
   }
   void setLength(double l){
     length=l;
   }
```

```
   double getWidth(){
     return width;
   }
   void setWidth(double w){
     width=w;
   }
   double getArea(){
     return width*length;
   }
   double getPerimeter(){
     return 2*(width+length);
   }
};
int main(){
  Rectangle r1(5,6);
  cout<<"width: "<<r1.getWidth()<<" Length: "<<r1.getLength()<<endl;
  cout<<"Area: "<<r1.getArea()<<" Perimeter: "<<r1.getPerimeter()<<endl;
  Rectangle r2;
  cout<<"width: "<<r2.getWidth()<<" Length: "<<r2.getLength()<<endl;
  cout<<"Area: "<<r2.getArea()<<" Perimeter: "<<r2.getPerimeter()<<endl;
  return 0;
}
```

```
width: 5 Length: 6
Area: 30 Perimeter: 22
width: 1 Length: 1
Area: 1 Perimeter: 4
```

Rectangle objects in memory



2- Student

- Create two objects with the values:
- S1→ 1, Ali, 60.
- S2→ 2, Mona, 80.

Student
- id: int - name: string - degree: double
+ Student() + Student(double) + Student(int, string, double) + getId(): int + setId(int):void + getName(): string + setName(string):void + getDegree():double + setDegree():void + computeGrade(): char

```
#include <iostream>
using namespace std;
class Student
{
  private:
   int id;
   string name;
   double degree;
  public:
   Student() {
    id= 1.0;
    name="Ali";
    degree=50;
   }
   Student(int i, string n, double d){
    id=i;
    name=n;
    degree=d;
   }
   int getId(){
     return id;
   }
   void setId(int i){
     id=i;
   }
```

```
    string getName(){
     return name;
    }
    void setName(string n){
      name=n;
    }
    double getDegree(){
      return degree;
    }
    void setDegree(double d){
      degree=d;
    }
    void computeGrade(){
      if(degree>=85)
        cout<<"Excellent";
      else if (degree>=75)
         cout<<"very good";
      else if(degree>=65)
         cout<<"good";
      else if(degree>=50)
         cout<<"Acceptable";
      else
        cout<<"fail";
    }
};
```

```
int main()
{
  Student s1;
  cout<<"Id: "<<s1.getId() <<" name: "<<s1.getName();
  cout<<" degree: "<<s1.getDegree ()<<" ";
  s1.computeGrade();
  Student s2(2, "Mona", 80);
  cout<<"\nId: "<<s2.getId() <<" name: "<<s2.getName();
  cout<<" degree: "<<s2.getDegree()<<" ";
  s2.computeGrade();
}
```

```
Id: 1 name: Ali degree: 50 Acceptable
Id: 2 name: Mona degree: 80 very good
```

Student objects in memory